Inference Optimization

Bottlenecks, latency and throughput, accelerators, model optimizations, and serving strategies

Optimize the bottleneck measured under realistic traffic. Faster arithmetic, smaller weights, better memory use, and better scheduling solve different problems; judge gains by successful work within latency and cost limits.

Source: Chip Huyen, AI Engineering: Building Applications with Foundation Models, first edition (2025). Source: book pp. 405–448 / PDF p. 429. These are original study notes, not verbatim chapter reproductions. Tool names, prices, model comparisons, and research findings reflect the book's publication period unless stated otherwise.

01 Understanding Inference Optimization

Source: book pp. 406–412 / PDF p. 430

Inference overview and computational bottlenecks

An inference service receives requests, queues/schedules work, runs model computation, and returns results. Observed latency includes networking, queueing, input processing, generation, and other application components, not just accelerator execution.

Compute-bound: arithmetic throughput limits speed. **Memory-bandwidth-bound:** moving data to compute units limits speed. **Capacity-bound:** needed tensors do not fit memory. These are different failure modes; "memory-bound" is sometimes used ambiguously. A roofline model relates arithmetic intensity (operations per byte moved) to achievable compute.

Prefill and decode

During **prefill**, input tokens are processed, largely in parallel, and initial key/value states are created. During **decode**, output tokens are generated step by step, reusing cached states. Prefill often has higher arithmetic intensity; small-batch decoding often spends more time moving weights and cache data than doing arithmetic.

Sequence lengths, batch size, model size, architecture, cache layout, and hardware change the balance. Treat "prefill compute-bound, decode bandwidth-bound" as a useful starting model, not an invariant for every deployment.

Online and batch inference APIs

Online serving prioritizes user latency; asynchronous batch APIs trade longer completion windows for higher throughput and potentially lower prices. A batch API processes submitted work later; it does not mean answers necessarily existed before requests arrived, as in some traditional precomputed prediction systems.

Internal batching can occur in either service. Choose based on urgency and deadlines, and compare the price and latency of the actual provider/version rather than assuming historical discounts are permanent.

---

02 Inference Performance Metrics

Source: book pp. 412–419 / PDF p. 436

Latency, TTFT, TPOT, and percentiles

TTFT: time until the first visible output token. **TPOT/ITL:** time per subsequent token. **Total latency:** time until response completion. A streaming interface changes perceived responsiveness but not necessarily completion time.

> Generation latency ≈ TTFT + $(N_{out} - 1) \times$ TPOT

This approximation assumes TPOT excludes the first token. Include pipeline/queueing overhead according to your measurement boundary. Report p50/p90/p95/p99 and length/concurrency distributions; means can hide severe tails. Hidden agent steps delay user-visible TTFT.

Throughput, goodput, and cost

Throughput counts work per time: requests/s or tokens/s. State whether tokens are input, output, or both, and whether throughput is per user or aggregate. **Goodput** counts work meeting the service-level objective, preventing fast-but-late requests from looking successful.

> Cost per 1M output tokens ≈ hourly serving cost × 1,000,000 / (3,600 × output tokens/s)

At $2/hour and 100 output tokens/s, the compute-only estimate is about $5.56 per million output tokens. Add input processing, idle capacity, networking, tools, retries, and operations. Batch throughput improvements can worsen individual latency.

Utilization, MFU, and MBU

A GPU showing "busy" in a monitoring tool need not be doing useful work near peak capacity. **Model FLOP utilization (MFU)** compares achieved useful arithmetic to relevant theoretical peak. **Memory bandwidth utilization (MBU)** compares achieved memory traffic rate to peak bandwidth.

Low utilization can reveal bottlenecks or overhead, but maximizing utilization is not the final objective. Compare quality, goodput, latency, and cost under equivalent workloads. Peak hardware numbers depend on precision, sparsity, and conditions; mismatched denominators make utilization misleading.

---

03 AI Accelerators

Source: book pp. 419–426 / PDF p. 443

Compute, memory hierarchy, power, and selection

CPUs emphasize general-purpose control and latency; GPUs and other AI accelerators emphasize parallel workloads. Consider dtype/kernel support, achievable compute, memory capacity, memory bandwidth, interconnects, power, cooling, availability, and software maturity.

Registers/on-chip memory are fast but small; device HBM holds larger tensors; host memory and off-device transfers add movement overhead. Some products emphasize low-latency inference, others throughput or training scale. Historical H100/TPU comparisons are not universal purchasing advice.

Choose hardware using representative model size, batch/sequence lengths, SLOs, and full operating cost. An unsupported operator or poor compiler/kernel path can negate attractive peak FLOP/s.

---

04 Model Optimization

Source: book pp. 426–440 / PDF p. 450

Compression: quantization, distillation, pruning

Quantization reduces bit width; distillation trains a smaller task-appropriate student; pruning removes selected weights, structures, or layers. Unstructured zeros do not automatically accelerate dense kernels; structured sparsity and matching hardware/software are necessary for many speed gains.

Compression often trades capacity or precision for lower storage/bandwidth needs. Re-evaluate the exact downstream workload, including rare cases and safety behavior. Smaller checkpoints do not always mean faster end-to-end inference.

Overcoming autoregressive decoding

Speculative decoding: a cheap draft proposes several tokens; a target model verifies them in parallel and accepts a valid prefix, correcting/rejecting as needed. Proper acceptance/rejection sampling can preserve the target distribution, unlike merely picking any plausible draft.

Gains depend on draft speed, acceptance rate, verification cost, and available compute. Poor drafts can add overhead. Reference-based drafting reuses likely continuations from an input/document. Parallel decoding approaches, including multi-head techniques such as Medusa and iterative methods, attempt several future tokens at once and may require extra training or different validation.

Attention optimization and KV-cache math

Autoregressive attention reuses prior keys and values instead of recomputing them at every step. The cache increases with active sequences, sequence length, layers, key/value dimensions, and dtype.

> $\text{KV bytes} = 2 \times B \times S \times L \times n_{\text{KV heads}} \times d_{\text{head}} \times \text{bytes/value}$

For ordinary multi-head attention, $n_{\text{KV heads}} d_{\text{head}}$ equals hidden dimension. Multi-query shares KV across all query heads; grouped-query shares within groups. These reduce storage, while sliding/local attention and cross-layer sharing change attention behavior or architecture.

Paged cache management reduces fragmentation; quantization/compression reduces bytes. Account for cache growth under concurrency and long contexts before sizing replicas.

Kernels and compilers

FlashAttention-style kernels reduce attention's memory traffic through tiling and fusion without simply approximating attention scores. Kernel optimizations include vectorization, parallelism, operator fusion, and minimizing redundant transfers/computation.

Compilers lower high-level operations to efficient hardware instructions and kernels. Examples include torch.compile, XLA, TVM, MLIR infrastructure, and TensorRT. Shape specialization, warm-up/compilation costs, numerical changes, and hardware compatibility affect practical results. Test quality as well as throughput after optimization.

---

05 Inference Service Optimization

Source: book pp. 440–448 / PDF p. 464

Static, dynamic, and continuous batching

Static batching waits for a group and can strand capacity behind long requests. Dynamic batching adds a maximum waiting window so requests do not wait indefinitely for a full batch. Padding and length variation still waste work.

Continuous/in-flight batching schedules at decoding iterations: completed sequences leave and new ones enter. It improves occupancy for variable output lengths but needs careful cache allocation and admission control. Test throughput and tail latency together; larger batches can worsen the user experience.

Disaggregated prefill/decode and prompt caching

Separate prefill and decode pools can match hardware/scheduling to their different resource profiles. The benefit must exceed state-transfer and network costs; workload lengths and TTFT/TPOT priorities determine allocation.

Prompt/prefix caching reuses computation for shared token prefixes, such as long system instructions or conversation history. It differs from response caching and from a single sequence's KV reuse, though implementations use related states. Stable exact prefixes, versioning, isolation, cache lifetime, and memory pressure determine effectiveness. Provider pricing and semantics vary.

Replica, tensor, and pipeline parallelism

Replica parallelism: independent model copies serve different requests; useful when a model fits per replica. **Tensor parallelism:** split operations/weight tensors across devices, enabling larger models and potentially reducing latency but adding synchronization. **Pipeline parallelism:** split layers/stages and overlap microbatches; communication and pipeline bubbles can hurt per-request latency.

Choose topology with model size, interconnect bandwidth, traffic, and SLOs in mind. Service changes often preserve model weights, but scheduling and numerical implementations can still affect behavior; run regression and load tests.

Revision Check

1. What would distinguish capacity pressure from bandwidth pressure?
2. Which changes improve TTFT versus TPOT?
3. Why can throughput rise while goodput falls?
4. How much KV memory does your expected concurrency require?
5. When will speculative decoding or tensor parallelism be counterproductive?

Measure first, optimize the dominant bottleneck, and remeasure quality, tails, goodput, and cost. Hardware peak figures and isolated token speed are not sufficient acceptance criteria.

Book-based notes, separate from the earlier transcript-based notes. Coverage is indexed by section in coverage.md. Source links open the supplied book at the relevant PDF page. Detailed examples, citations, and complete experimental evidence remain in the book.